

Mapping Reductions, Continued: Showing that ALL_{TM} is Not Co-recognizable

Last week we introduced mapping reductions, and we'll do a bit more this week. Let's start with ALL_{TM} .

We've guessed that ALL_{TM} is neither recognizable nor co-recognizable.

Once again, non-co-recognizability is easier to prove in this case, so let's do that first.

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

M_0 = “On input x :

1. Pass.

2. Run M on w .

3.

2. Return $\langle M_0 \rangle$.”

- Idea: If we pass on step 1 and M loops on w , then M_0 won't accept anything (and in particular, it won't accept everything). So it won't be in ALL_{TM} .

Showing that ALL_{TM} is Not Co-recognizable (2)

Now, let's do the mapping reduction proof:

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

$M_0 =$ “On input x :

1. Pass.

2. Run M on w .

3. *Accept*.”

2. Return $\langle M_0 \rangle$.”

Proof that $\langle M_0 \rangle \in \text{ALL}_{\text{TM}}$ iff $\langle M, w \rangle \in \text{HALT}$:

($\Leftarrow$) Suppose that $\langle M, w \rangle \in \text{HALT}$.

Then M_0 always gets to line 2.

When it does, M will halt, and so M_0 will reach line 3 and accept.

So M_0 accepts every input.

$\Rightarrow \langle M_0 \rangle \in \text{ALL}_{\text{TM}}$.

($\Rightarrow$) Suppose that $\langle M, w \rangle \notin \text{HALT}$.

Then M_0 always gets to line 2.

When it does, M will loop, and so M_0 will never reach line 3.

So M_0 doesn't accept any input (and so doesn't accept every input, either).

$\Rightarrow \langle M_0 \rangle \notin \text{ALL}_{\text{TM}}$.

Showing that ALL_{TM} is Not Recognizable

This one is actually pretty tricky:

- We want M_0 to accept every string iff M loops on w .
- Last time we said that step 2 acts as a filter: If M loops we don't get to step 3, so any string we would accept at that point is filtered out.
- Idea: We don't want a filter, so we need to change step 2.

Think back to A_{TM} : If M loops, how many k will be valid certificates?

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

M_0 = “On input x :

1. Pass.

- 2.

- 3.

2. Return $\langle M_0 \rangle$.”

Showing that ALL_{TM} is Not Recognizable (2)

Now, let's do the mapping reduction proof:

- Consider the TM P “On input $\langle M, w \rangle$:

- Construct the TM M_0 as follows:

M_0 = “On input $\langle k \rangle$:

- Pass.
- Run M on w for k steps.
- If it halts, *loop*, otherwise *accept*.”

- Return $\langle M_0 \rangle$.”

Proof that $\langle M_0 \rangle \in \text{ALL}_{\text{TM}}$ iff $\langle M, w \rangle \in \overline{\text{HALT}}$:

($\Leftarrow$) Suppose that $\langle M, w \rangle \in \overline{\text{HALT}}$.

Then M_0 always gets to line 2.

When it does, M not halt within k steps for any input k , since it loops.

So M_0 accepts every input.

$\Rightarrow \langle M_0 \rangle \in \text{ALL}_{\text{TM}}$.

($\Rightarrow$) Suppose that $\langle M, w \rangle \notin \overline{\text{HALT}}$.

Then M_0 always gets to line 2.

When it does, M will halt for some input k .

For this k , M_0 will loop, and so it will not accept.

$\Rightarrow \langle M_0 \rangle \notin \text{ALL}_{\text{TM}}$.

Recognizability and Co-recognizability, Continued...

There are times when we want to reduce from a language other than HALT. Specifically, let's show that $L_2 \leq_m \text{ALL}_{\text{TM}}$:

- 1 If we do this reduction, what will our input be?

An instance of L_2 , i.e., a TM description $\langle M \rangle$.

- 2 What will our output be?

An instance of ALL_{TM} , i.e., a TM description $\langle M_0 \rangle$.

Since we're not using the input $\langle M, w \rangle$ we can't do the default reduction: line 2 won't have any meaning without giving a w to run.

Reducing from Something Other than HALT

What we want to do is output a TM M_0 that halts on every input iff the original M accepts only the input 1 — that is, if:

- If M sees an input 1 it accepts, and
- If M sees any other input it doesn't accept.

This means that we want M_0 to do two things:

- We want it to fail to accept every input if M doesn't accept 1, and
- We want there to some inputs that it does not accept in the event that M accepts anything other than 1.

How could we program a TM to do these two things?

- Start by running M on 1 — if it loops M_0 will get stuck there and won't accept anything.
- Use the idea from ALL_{TM} earlier and dovetail over all inputs other than 1 for the TM M .

Reducing from Something Other than HALT (2)

Here's a reduction:

Let $w_0, w_1, \dots$ be an effective enumeration of Σ^* .

- Consider the TM P “On input $\langle M \rangle$:

1. Construct the TM M_0 as follows:

$M_0 =$ “On input $\langle x, k \rangle$:

1. Run M on 1.
2. If it rejects, *loop*. Otherwise,
3. If $x \neq 1$:
4. Run M on x for k steps.
6. If it accepts, *loop*.
7. *Accept*. ”

2. Return $\langle M_0 \rangle$.”

Reducing from Something Other than HALT (3)

($\Leftarrow$) Suppose that $\langle M \rangle \in L_2$.

Then M accepts only the input 1.

So M will accept 1 in step 1, and M_0 will get to step 2 and 3.

When it does, M not accept within k steps for any input $x \neq 1$, since M only accepts 1.

So M_0 accepts every input.

$\Rightarrow \langle M_0 \rangle \in \text{ALL}_{\text{TM}}$.

($\Rightarrow$) Suppose that $\langle M \rangle \notin L_2$.

Then M either does not accept 1 or it accepts some other string.

If M does not accept 1, M_0 will loop on either line 1 or 2 (and not accept any input).

If M does accept 1, then there is some string $x \neq 1$ that it also accepts.

Let k be the number of steps it takes to do so.

On input $\langle x, k \rangle$, M will loop.

$\Rightarrow M_0$ will not accept this $\langle x, k \rangle$.

$\Rightarrow \langle M_0 \rangle \notin \text{ALL}_{\text{TM}}$.

More Examples

Let's look at a few new languages:

- $\text{EQ}_{\text{TM}} = \{ \langle M, N \rangle \mid L(M) = L(N) \}$
- $L_3 = \{ \langle M \rangle \mid \exists y, M \text{ outputs } y + 5 \text{ on the input } \langle y \rangle \}$
- $\text{DEC} = \{ \langle M \rangle \mid L(M) \text{ is decidable} \}$
- $\text{FIN} = \{ \langle M \rangle \mid |L(M)| < \infty \}$

To start:

- Do you think that $\text{EQ}_{\text{TM}} = \{ \langle M, N \rangle \mid L(M) = L(N) \}$ is recognizable?
Co-recognizable?
- What about L_3 ?
- What about DEC?
- What about FIN?

Proof that EQ_{TM} is not recognizable

The trick for this reduction: the output this time isn't just a single $\langle M \rangle$, but a pair $\langle M, N \rangle$.

We want to reduce from $\overline{\text{HALT}}$ and output two TMs that will give the same language iff M loops on w .

- Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

M_0 = “On input x :

1. Pass.
2. Run M on w .
3. *Accept.* ”

2. Construct the TM N_0 as follows:

N_0 = “On input x :

1. *Reject.* ”

3. Return $\langle M_0, N_0 \rangle$.”

Proof that EQ_{TM} is not recognizable (2)

Proof that $\langle M_0, N_0 \rangle \in \text{EQ}_{\text{TM}}$ iff $\langle M, w \rangle \in \overline{\text{HALT}}$:

($\Leftarrow$) Suppose that $\langle M, w \rangle \in \overline{\text{HALT}}$.

Then M loops on w .

Then M_0 will loop on step 2 and will not reach step 3.

So $L(M_0) = \emptyset = L(N_0)$.

$\Rightarrow \langle M_0, N_0 \rangle \in \text{EQ}_{\text{TM}}$.

($\Rightarrow$) Suppose that $\langle M, w \rangle \notin \overline{\text{HALT}}$.

Then M halts on w .

So M_0 will complete step 2 and get to step 3.

When it does, it will accept.

So $L(M_0) = \Sigma^* \neq L(N_0)$.

$\Rightarrow \langle M_0, N_0 \rangle \notin \text{EQ}_{\text{TM}}$.

Proof that EQ_{TM} is not co-recognizable

The idea here is similar to the last time, but we're reducing from $HALT$.

The nice thing about this language is that we can almost reuse the original reduction:

- Consider the TM P “On input $\langle M, w \rangle$:
 1. Construct the TM M_0 as follows:
 - $M_0 =$ “On input x :
 1. Pass.
 2. Run M on w .
 3. *Accept.*”
 2. Construct the TM N_0 as follows:
 - $N_0 =$ “On input x :
 1. *Accept.*”
 3. Return $\langle M_0, N_0 \rangle$.”

Proof that EQ_{TM} is not co-recognizable (2)

Proof that $\langle M_0, N_0 \rangle \in \text{EQ}_{\text{TM}}$ iff $\langle M, w \rangle \in \text{HALT}$:

($\Leftarrow$) Suppose that $\langle M, w \rangle \in \text{HALT}$.

Then M halts on w .

So M_0 will complete step 2 and get to step 3.

When it does, it will accept.

So $L(M_0) = \Sigma^* = L(N_0)$.

$\Rightarrow \langle M_0, N_0 \rangle \in \text{EQ}_{\text{TM}}$.

($\Rightarrow$) Suppose that $\langle M, w \rangle \notin \text{HALT}$.

Then M loops on w .

Then M_0 will loop on step 2 and will not reach step 3.

So $L(M_0) = \emptyset \neq L(N_0)$.

$\Rightarrow \langle M_0, N_0 \rangle \notin \text{EQ}_{\text{TM}}$.

Showing L_3 is recognizable

This is actually straightforward — I'm putting this in as an example so you can see a reduction that uses a TM with an output other than TRUE/FALSE.

The certificate we'll give will be:

$\langle y, k \rangle$, where y is the input that results in the output $y + 5$, and k is the time M takes to halt.

An enumerator:

“On input $\langle M \rangle$:

1. For $k = 0$ to ∞ :
2. For $y = 0$ to k :
3. Run M on $\langle y \rangle$ for k steps.
4. If it halts and outputs $y + 5$, *accept*.

Showing L_3 is not co-recognizable

We'll use something like the default reduction again, but this time we'll change our M_0 so that it outputs a value:

- Consider the TM P “On input $\langle M, w \rangle$:
 1. Construct the TM M_0 as follows:
 $M_0 =$ “On input $\langle y \rangle$:
 1. Pass
 2. Run M on w .
 3. Output $y + 5$. ”
 2. Return $\langle M_0 \rangle$.”

Showing L_3 is not co-recognizable (2)

Proof that $\langle M_0 \rangle \in L$ iff $\langle M, w \rangle \in \text{HALT}$:

$(\Leftarrow)$ Suppose that $\langle M, w \rangle \in \text{HALT}$.

Then M halts on w .

So M_0 will get to step 2 and 3.

When it does, it will output $y + 5$.

Since M_0 outputs $y + 5$ for every input, there exists at least one input for which it outputs $y + 5$ (e.g., $y = 1$).

$\Rightarrow \langle M_0 \rangle \in L_3$.

$(\Rightarrow)$ Suppose that $\langle M, w \rangle \notin \text{HALT}$.

Then M loops on w .

Therefore M_0 loops on line 2.

So for every input, M_0 loops and does not output anything (in particular, it never outputs $y + 5$).

$\Rightarrow \langle M_0 \rangle \notin L_3$.

Showing that DEC is not recognizable

We want to set up the reduction so that $L(M_0)$ is recognizable but not decidable if M halts on w . We'll have $L(M_0) = \text{HALT}$ in this case.

Can you see why we say that $L(M_0)$ has to be recognizable?

So M_0 should take HALT instances as input — i.e., TM/input pairs. To distinguish this pair from $\langle M, w \rangle$, we'll refer to it as $\langle N, x \rangle$.

- Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

$M_0 =$ “On input $\langle N, x \rangle$:

1. Run N on x
2. Run M on w .
3. *accept.* ”

2. Return $\langle M_0 \rangle$.”

Showing that DEC is not recognizable (2)

Proof that $\langle M_0 \rangle \in \text{DEC}$ iff $\langle M \rangle \in \overline{\text{HALT}}$:

$(\Leftarrow)$ Suppose that $\langle M \rangle \in \overline{\text{HALT}}$.

Then M loops on w .

So M_0 loops on either step 1 or step 2.

$\Rightarrow L(M_0) = \emptyset$.

$\Rightarrow \langle M_0 \rangle \in \text{DEC}$.

$(\Rightarrow)$ Suppose that $\langle M \rangle \notin \overline{\text{HALT}}$.

Then M halts on w .

Therefore M_0 passes line 2 and accepts iff it finishes step 1.

So M_0 halts and accepts iff N halts on x .

$\Rightarrow L(M_0) = \text{HALT}$.

$\Rightarrow \langle M_0 \rangle \notin \text{DEC}$.

Showing that DEC is not co-recognizable

The idea this time is similar, but this time we'll use dovetailing.

Remember that we don't generally use dovetailing in the reduction program P , since P has to halt on all inputs. But we can make P write a program that dovetails.

- Consider the TM P “On input $\langle M, w \rangle$:
 1. Construct the TM M_0 as follows:
 $M_0 =$ “On input $\langle N, x \rangle$:
 1. For $k = 0$ to ∞ :
 2. Run N on x for k steps.
 3. Run M on w for k steps.
 4. If either halts, *accept*. ”
 2. Return $\langle M_0 \rangle$.”

Showing that DEC is not co-recognizable (2)

Proof that $\langle M_0 \rangle \in \text{DEC}$ iff $\langle M \rangle \in \text{HALT}$:

($\Leftarrow$) Suppose that $\langle M \rangle \in \text{HALT}$.

Then there is a k such that M halts on w in k steps.

So M_0 will accept by the k^{th} loop iteration, regardless of whether N halts on x .

$\Rightarrow L(M_0) = \Sigma^*$.

$\Rightarrow \langle M_0 \rangle \in \text{DEC}$.

($\Rightarrow$) Suppose that $\langle M \rangle \notin \text{HALT}$.

Then M loops on w .

So M does not halt on w in k steps for any k .

Therefore M_0 accepts iff there is a k for which N accepts x within k steps.

So M_0 halts and accepts iff N halts on x .

$\Rightarrow L(M_0) = \text{HALT}$.

$\Rightarrow \langle M_0 \rangle \notin \text{DEC}$.

The Language FIN: A Reduction From $\overline{ALL_{TM}}$

To start, let's look at how to write FIN using quantifiers:

Let $x_0, x_1, \dots$ be an effective enumeration of Σ^* . Then,

$$FIN = \{ \langle M \rangle \mid \exists i \in \mathbb{N}, \forall n > i, k \in \mathbb{N}, \langle M, x_n, k \rangle \notin ACC \}.$$

Here we're making use of the idea that if a language is finite, then it has an element that is latest in the ordering of Σ^ . If x_{n_0} is this last element, then its index n_0 is an i that'll work in this definition.*

We'll argue that this language is neither recognizable nor co-recognizable, so we'll prove it by reducing from $\overline{ALL_{TM}}$.

- *Why $\overline{ALL_{TM}}$ and not ALL_{TM} ? Because of the quantifiers!*

$$\overline{ALL_{TM}} = \{ \langle M \rangle \mid \exists x, \forall k, \langle M, x, k \rangle \notin ACC \}.$$

The Language FIN: A Reduction From $\overline{\text{ALL}}_{\text{TM}}$ (2)

Here's our reduction:

- Let $x_0, x_1, \dots$ be an effective enumeration of Σ^* .

Consider the TM P “On input $\langle M \rangle$:

1. Construct the TM M_0 as follows:

M_0 = “On input w :

1. For $i = 0$ to ∞ :
2. Run M on x_i .
3. If it rejects, *loop*.
4. If $w == x_i$, *accept*.”

2. Return $\langle M_0 \rangle$.”

The Language FIN: A Reduction From $\overline{\text{ALL}_{\text{TM}}}$ (3)

Proof that $\langle M_0 \rangle \in \text{FIN}$ iff $\langle M \rangle \in \overline{\text{ALL}_{\text{TM}}}$:

$(\Leftarrow)$ Suppose that $\langle M \rangle \in \overline{\text{ALL}_{\text{TM}}}$.

Then M does not accept $x_{i'}$ for some $i' \in \mathbb{N}$.

Then for any $n > i'$, if we feed x_n into M_0 it will either loop before hitting iteration $i = i'$, or it will eventually hit iteration $i = i'$.

When it does, it will loop on step 3.

So M_0 will not accept any $x_n, n > i'$.

So $|L(M_0)| < \infty$.

$\Rightarrow \langle M_0 \rangle \in \text{FIN}$.

$(\Rightarrow)$ Suppose that $\langle M \rangle \notin \overline{\text{ALL}_{\text{TM}}}$.

Then M accepts every string w .

If we feed any string x_n into M_0 , M_0 will run M on every $x_i, i \leq n$.

When it does, M will eventually accept all of these strings, and so M_0 will accept.

So $|L(M_0)| = \infty$.

$\Rightarrow \langle M_0 \rangle \notin \text{FIN}$.

More Practice: A Reduction To $\overline{\text{ALL}}_{\text{TM}}$

As practice, we can also reduce FIN to $\overline{\text{ALL}}_{\text{TM}}$:

We'll use the same property as before: if a $L(M)$ is infinite then, for any j' , there is an $n > j'$ such that M accepts x_n .

- Let $x_0, x_1, \dots$ be an effective enumeration of Σ^* .

Consider the TM P “On input $\langle M \rangle$:

1. Construct the TM M_0 as follows:

M_0 = “On input j :

1. For $i = j$ to ∞ :
2. For $n = j$ to i :
3. Run M on x_n for i steps.
4. If it accepts, *accept*.

2. Return $\langle M_0 \rangle$.”

More Practice: A Reduction To $\overline{\text{ALL}_{\text{TM}}}$ (2)

Proof that $\langle M_0 \rangle \in \overline{\text{ALL}_{\text{TM}}}$ iff $\langle M \rangle \in \text{FIN}$:

($\Leftarrow$) Suppose that $\langle M \rangle \in \text{FIN}$.

Then $L(M) < \infty$.

So there is some $j \in \mathbb{N}$, $n > j \rightarrow M$ does not accept x_n .

If we feed $j + 1$ into M_0 , it will loop only on values of $n > j$, and so when it runs M on x_n on line 3, M will never accept.

So M_0 will never accept j .

$\Rightarrow \langle M_0 \rangle \in \overline{\text{ALL}_{\text{TM}}}$.

($\Rightarrow$) Suppose that $\langle M \rangle \notin \text{FIN}$.

Then $L(M) = \infty$.

So for any $j \in \mathbb{N}$, there is some $n > j$ such that M accepts x_n .

Therefore, if we feed any j into M_0 , there will be an n' such that M accepts $x_{n'}$ in some number, let's say, k , of steps.

If M_0 reaches the loop iterations $i = \max(k, n')$, $n = n'$, it will accept on line 4.

If it doesn't reach this loop iteration, it must have already accepted.

So M_0 accepts all j .

$\Rightarrow \langle M_0 \rangle \notin \overline{\text{ALL}_{\text{TM}}}$.

Rice's Theorem

You'll have noticed that all of the results we have so far are languages that deal with TMs.

This isn't too surprising: we got our first undecidability result by using TMs that use self-reference, and we've just built up from there. In fact, just about any statement about a TM's language is undecidable:

Rice's Theorem: (text, p.241)

Suppose that P is a language of TM descriptions such that:

- P is neither empty nor the set of all TM descriptions.
- If M_1 and M_2 are TMs such that $L(M_1) = L(M_2)$, then $\langle M_1 \rangle \in P$ iff $\langle M_2 \rangle \in P$.

Then P is undecidable.

Rice's Theorem (2)

The language P has to be defined in terms of the *semantics* of the TM, not by its *structure*, though — this is why we can tell that a C program will or will not compile, even if we can't say what it will do.

But you can see that this theorem means that many of the interesting questions we can ask about TMs (or, by extension, computer programs) will in general be undecidable.

Note: *Since part of the purpose of this course is to get you used to doing reductions,*
you are not allowed to use Rice's theorem in assignments or tests unless otherwise stated. *You can use it, however, as a quick way of proving undecidability for when you're outside of CSCC63.*

There's More than Just Recognizability and Co-Recognizability...

- Later on in the course we'll run into problems that we think cannot be solved efficiently, but which we have not yet proven to be intractable. What would be the consequences if these problems were solvable?
- We've proven in class that there's no TM that'll solve **HALT** for us. According to the Church-Turing thesis this means that there's no way to solve it at all using any reasonable computer.

But suppose that tomorrow someone comes up with a very surprising announcement: a bunch of new physical laws have been discovered which will allow us to build computers that solve **HALT**! How would this change the computability landscape?

Oracles are a way of analyzing these sorts of problems.

What is an Oracle?

An *Oracle* is...

- A theoretical object that can solve a single given language (like HALT).
- Something that can be plugged into a computer or TM to give it more power.
- Used for theory and thought experiments (not a real device).

You can treat them as magical boxes or crystal balls that you can plug into your computer.

The Church-Turing thesis suggests that these can't be built in the real world, but if they could, they would allow me to solve computational problems that I couldn't normally answer with a TM.

In particular, though, it is not a TM itself.

Using an Oracle

Now, the oracle on its own isn't going to do that much for us, unless we've got a lot of HALT instances we want to solve.

But if we plug it into our computer we'll be able to incorporate it into our programs, and that'll let us solve other problems as well:

Oracle program to solve A_{TM} :

Let $M_A^{\text{HALT}} = \text{"On input } \langle M, w \rangle \text{:}$

1. If M halts on w (use the oracle here):
2. Run M on w .
3. If it accepts, *accept*.
4. *Reject*."

We Can Solve Some Surprising Problems This Way...

Oracle program to solve $L_2 = \{\langle M \rangle \mid L(M) = \{1\}\}$:

Let $w_0, w_1, \dots$ be an effective enumeration of Σ^* .

Let $M_L^{\text{HALT}} = \text{"On input } \langle M \rangle \text{:}$

1. Let $M_1 = \text{"On input } x \text{:}$

1. Run M on 1.

2. If it accepts, *accept*, otherwise *loop*."

2. Let $M_2 = \text{"On input } x \text{:}$

1. For $k = 0$ to ∞ :

2. For $i = 0$ to k :

3. If $w_i \neq 1$:

4. Run M on w_i for k steps.

5. If it accepts, *accept*."

3. If M_1 halts on ε and If M_2 does not halt on ε (use the oracle here), *accept*.
Otherwise, reject."

...But Not All Of Them

We'd actually find that ALL_{TM} would be co-recognizable but not recognizable for these programs.

More generally, what would the oracle do if I fed it this program (with any input):

Let $M^{HALT} =$ “On input $\langle M, w \rangle$:

1. If M halts on w (use the oracle here):
2. *Loop.*
3. Else:
4. *Loop.*”

It would assume the program doesn't have access to an oracle — so the check on line 1 would lead to undefined behaviour.

In fact, all of the arguments we used at the beginning of the course to show A_{TM} and $HALT$ would apply here, too! The oracle machine can't solve its own halting problem.

Some Notation

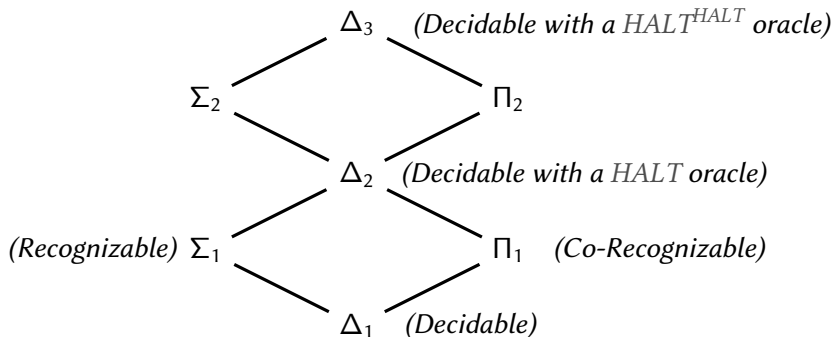
- You can see that we've denoted the program we've written with oracle access by using the $^{\text{HALT}}$ superscript. That's there to indicate that we're looking at a program that uses a bit more power than we'd normally have.
- It tells me that the “If M halts/loops on w ” conditions aren't mistakes, but are allowed because we're assuming we have a widget that'll answer the question for us.
 - TMs/computer programs without superscripts are regular programs that we can build on real computers: e.g., $M, N, \dots$
 - TMs/computer programs with superscripts are programs that have access to an oracle for whichever language is in the superscript: e.g., M^{HALT} can condition on HALT , while N^{ALLTM} can condition on ALLTM .
- We can do the same thing with languages: the superscript indicate that the programs in the language use an oracle. E.g.:

$$\text{HALT}^{\text{HALT}} = \{ \langle M^{\text{HALT}}, w \rangle \mid M^{\text{HALT}} \text{ halts on } w \}.$$

The Arithmetical Hierarchy

We find that using these oracles lets us separate the undecidable languages into levels: things that are decidable/recognizable/co-recognizable with an oracle for HALT , with an oracle for $\text{HALT}^{\text{HALT}}$, and so on...

...



Looking Back at our Examples

We've used certificates as a way of talking about recognizable languages, but can the Arithmetical Hierarchy help us generalize this concept?

Sometimes, yes — remember how we wrote our examples last week in set notation:

- $E_{TM} = \{ \langle M \rangle \mid \forall w \in \Sigma^*, k \in \mathbb{N}, \langle M, w, k \rangle \notin ACC \}$.
This is co-recognizable.
- $L_1 = \{ \langle M \rangle \mid \exists \langle N, k \rangle, \langle M, N, k \rangle \in ACC \wedge \langle N, \varepsilon k \rangle \in ACC \}$.
This is recognizable.
- $L_2 = \{ \langle M \rangle \mid \forall w \neq 1, k, \exists k', \langle M, 1, k' \rangle \in ACC \wedge \langle M, w, k \rangle \notin ACC \}$
 $= \{ \langle M \rangle \mid \exists k', \forall w \neq 1, k, \langle M, 1, k' \rangle \in ACC \wedge \langle M, w, k \rangle \notin ACC \}$
Languages like L_2 are Δ_2 — they're both Σ_2 and Π_2 .
- $ALL_{TM} = \{ \langle M \rangle \mid \forall w \in \Sigma^*, \exists k \in \mathbb{N}, \langle M, w, k \rangle \in ACC \}$
This is in Π_2 .

But be careful — there are languages that can't be expressed this way!

What about questions not involving TMs?

One of the ideas we want you to get from this course is that undecidable programs can show up in many places, even if there is no obvious TM in the problem — if we look around (say, on YouTube), we can find TMs implemented in many different media: e.g., Lego, wooden machines, Minecraft, and so on.

So how do we prove undecidability for these sorts of problems?

If we can emulate a TM in using Lego/Minecraft/..., what does this tell us about the complexity of Lego/Minecraft/...?

If we can emulate a TM, we can express HALT in Lego/Minecraft/....

This is basically a mapping reduction!

If we can embed a TM into a system, some questions about that system will be undecidable.

Some important undecidable systems

1. The **Post Correspondence Problem (PCP)**: We'll go over this problem in detail next week, but you've seen it in tutorials.
2. General queries about cellular automata such as Conway's Game of Life,
3. Tiling problems on the plane (if you play board games, think generalized Carcassonne),
4. If you've taken group theory in math, the word problem on free groups is undecidable,

And many more...

Next week, we'll go over a more complicated version of what you've done in tutorials — we'll embed **HALT** into the PCP, and so show that it is undecidable.